BMET5933 Assignment 2 — Demo Summary

Deep Learning Technical Contribution · James Wu

This is the slim demo notebook for the kidney CT classification project. It mirrors my PowerPoint deck section-by-section so I can fall back to it if I am not allowed to use slides during the demo. I keep only the meaningful code snippets — full implementations and exhaustive analyses live in `James_Wu_Deep_Learning_Technical_Contribution.ipynb` and in the `Planning/` vault.

Project: 4-class kidney CT classification (Cyst / Normal / Stone / Tumor) on the Islam et al. 2022 benchmark. **Team**: Anthony Lee (classical handcrafted features + SVM) and James Wu (me, transfer-learned CNNs). **My contribution**: the entire deep-learning pipeline plus the shared preprocessing and the leakage diagnostic.

1. Shared infrastructure (what both pipelines use)

Both Anthony's classical SVM and my deep-learning models share the same data loader and the same evaluation harness. I designed these jointly with Anthony in Phase 0 so the classical-vs-DL comparison is apples-to-apples.

The key shared building block is `load_image()` — every CT slice enters the system through this function before any pipeline-specific work happens.

In [ ]:
```
# Section 1 — shared image loader. Used by both pipelines.

# `PIL` = Python Imaging Library (fork "Pillow"). `Image` is the submodule
# that gives us .open(), .convert(), .crop(), .resize() etc.
from PIL import Image

# `numpy` is aliased as `np`. Used to return the final image as a
# uint8 array — downstream pipelines expect numpy, not PIL objects.
import numpy as np

# `IMAGE_SIZE` is a project-wide constant defined in shared/config.py.
# Its value is the tuple (256, 256) — the canonical working resolution.
from shared.config import IMAGE_SIZE


# `def` defines a function. `load_image` is the function name.
# `path` is a positional parameter: filesystem path (str or Path).
def load_image(path):
    # `Image.open(path)` lazily opens the file, returns a PIL Image.
```

```
    # "Lazy" = pixels not read from disk until .load()/.convert().
    # `.convert("L")` converts the image mode to "L" = 8-bit single-channel
    # luminance (greyscale). CT scans are intrinsically intensity; we don't
    # want to fake RGB at this stage.
    img = Image.open(path).convert("L")

    # `.size` is a (width, height) tuple of the PIL Image.
    # Tuple-unpacking assigns the two ints to local names `w` and `h`.
    w, h = img.size

    # `min(w, h)` is the Python built-in: pick the smaller of the two.
    # `m` will become the side length of the square we crop out.
    m = min(w, h)

    # Compute the (left, top) corner so the crop is centered on the image.
    # Integer division `//` gives the floor; `.crop` needs int coords.
    left = (w - m) // 2
    top = (h - m) // 2

    # `.crop((left, top, right, bottom))` returns a new PIL Image with
    # the rectangle from (left, top) to (left+m, top+m). 4-tuple is a PIL
    # convention; coordinates are pixel positions, not percentages.
    img = img.crop((left, top, left + m, top + m))

    # `.resize(size, resample)` returns a PIL Image at the new size.
    # `IMAGE_SIZE` = (256, 256). `Image.BILINEAR` is the resampling filter:
    # weighted avg of 4 nearest pixels; smooth and edge-preserving;
    # standard for medical imaging.
    img = img.resize(IMAGE_SIZE, Image.BILINEAR)

    # `np.asarray(img, dtype=np.uint8)` converts the PIL Image into a numpy
    # array with shape (H, W) = (256, 256) and dtype uint8 (values 0-255).
    # `asarray` (vs `array`) avoids copying when already an ndarray.
    return np.asarray(img, dtype=np.uint8)
```

Why this matters for my demo: the deep-learning input pipeline and the classical feature-extraction pipeline both start from the same 256×256 grayscale uint8 array. Differences from here on are intentional, not accidents of preprocessing.

2. Dataset and split

After deduplication (more on this in Section 8), the benchmark contains 11,929 unique CT slices across four classes. I work from the same `split_full.csv` that Anthony's pipeline uses — train/val/test = 8,146 / 1,895 / 1,888, stratified by class with patient-group buckets so adjacent slices from the same patient stay together.

In [ ]:
```
# Section 2 — confirm the split I report against.

# `pandas` is aliased as `pd`. Standard tabular-data library; we
# use it to read the CSV and build the summary table.
import pandas as pd
# `pathlib.Path` is Python's modern OO path API — replaces os.path string
```

```
# manipulation with operator-overloaded path objects.
from pathlib import Path

# `Path.cwd()` returns the current working directory as a Path object.
ROOT = Path.cwd()

# If the notebook is opened from inside notebooks/, hop one level up.
# `(ROOT / "split_full.csv")` uses the `/` operator that Path overloads to
# join path components — equivalent to os.path.join.
# `.exists()` returns True/False without touching the file contents.
# `ROOT.parent` is the parent directory (one level up).
if (not (ROOT / "split_full.csv").exists()
        and (ROOT.parent / "split_full.csv").exists()):
    ROOT = ROOT.parent

# `pd.read_csv(path)` reads a CSV file into a pandas DataFrame.
# The DataFrame has columns "filename", "class", "split".
split = pd.read_csv(ROOT / "split_full.csv")

# Build the per-(split, class) count summary.
# `.groupby(["split", "class"])` groups rows by both columns (multi-index).
# `.size()` returns a Series of counts per group.
# `.unstack(fill_value=0)` pivots the "class" level into columns; any
# missing (split, class) combination becomes 0 instead of NaN.
# `.reindex(columns=[...])` enforces the canonical column order so Cyst
# always comes first regardless of pandas's internal alphabetical sort.
summary = (
    split.groupby(["split", "class"]).size()
    .unstack(fill_value=0)
    .reindex(columns=["Cyst", "Normal", "Stone", "Tumor"])
)

# Add a "Total" column = sum across each row (axis=1 means row-wise sum).
summary["Total"] = summary.sum(axis=1)

# `print()` writes the DataFrame to stdout. Default formatting is
# space-aligned columns, fine for a console demo.
print(summary)
```

I deliberately do not touch the held-out test set during training or checkpoint selection — every test number I report is from a single inference pass on the final model.

3. Architecture choice — two CNN backbones

I picked two backbones for the deep-learning side: **EfficientNet-B0** (5M parameters, my baseline) and **ConvNeXt V2 Base** (89M parameters, my primary model). They share the same data, same protocol, same evaluation harness, so the comparison isolates the architecture-and-pretraining effect.

I chose these specifically because:

- EfficientNet-B0 is small enough to fit comfortably in memory and trains fast, giving me a solid lightweight baseline.
- ConvNeXt V2 Base has ImageNet-22k → 1k pretraining and is a modern CNN that competes with vision transformers — I wanted to test whether more capacity and stronger pretraining help on this benchmark.

In [ ]:

```python
# Section 3 — model builders. Single dispatcher style so I can swap
# backbones with one CLI flag (--model efficientnet_b0 or convnextv2_base).

# `torch.nn` is the PyTorch submodule for neural-network layers (Linear,
# Conv2d, Dropout, Sequential, ...). `nn` is the conventional alias.
import torch.nn as nn

# `torchvision.models` ships pretrained CNN architectures. We import the
# constructor `efficientnet_b0` and weight-enum `EfficientNet_B0_Weights`
# that maps a string ("IMAGENET1K_V1") to a pretrained checkpoint.
from torchvision.models import EfficientNet_B0_Weights, efficientnet_b0

# `timm` (PyTorch Image Models) is the standard library for modern
# vision backbones with pretrained weights. We use it for ConvNeXt V2.
import timm


# `def` declares a function. `build_efficientnet_b0` returns a model ready
# for fine-tuning. `num_classes=4` is a keyword argument with default 4.
def build_efficientnet_b0(num_classes=4):
    # `EfficientNet_B0_Weights.IMAGENET1K_V1` selects the v1 ImageNet-1k
    # pretrained weights checkpoint provided by torchvision.
    weights = EfficientNet_B0_Weights.IMAGENET1K_V1

    # `efficientnet_b0(weights=weights)` instantiates the architecture and
    # downloads/applies the pretrained weights to every layer including the
    # original 1000-class ImageNet head.
    model = efficientnet_b0(weights=weights)

    # `model.classifier` is the final classifier head — a Sequential
    # containing [Dropout, Linear(in_features, 1000)].
    # `[1]` indexes the Linear layer; `.in_features` reads its input dim
    # (1280 for EfficientNet-B0).
    in_features = model.classifier[1].in_features

    # Replace the 1000-class head with our own 4-class head.
    # `nn.Sequential` chains layers in order.
    # `nn.Dropout(p=0.3)` randomly zeros 30% of activations in training
    # for regularisation; turned off automatically during model.eval().
    # `nn.Linear(in_features, num_classes)` is the final fully-connected
    # layer mapping 1280-dim features to our 4 logits.
    model.classifier = nn.Sequential(
        nn.Dropout(p=0.3),
        nn.Linear(in_features, num_classes),
    )

    # Return the modified model (still has the pretrained backbone weights;
    # only the head is freshly initialised).
```

```
    return model


# Builder for the larger backbone. `image_size=384` is our default because
# ConvNeXt V2 was pretrained at 384.
def build_convnextv2_base(num_classes=4, image_size=384):
    # Pick the matching pretrained checkpoint by name. timm uses dotted
    # of the form "<arch>.<pretrain_tag>".
    # "fcmae_ft_in22k_in1k_384" decodes as:
    #   fcmae        = Fully Conv. Masked AutoEncoder (self-supervised
    #                  pretraining used by ConvNeXt V2)
    #   ft_in22k     = fine-tuned on ImageNet-22k (~14M imgs, 22k cls)
    #   in1k         = fine-tuned on ImageNet-1k (1.28M imgs, 1000 cls)
    #   _384         = final input resolution 384x384
    name = (
        "convnextv2_base.fcmae_ft_in22k_in1k_384"
        if image_size >= 384
        else "convnextv2_base.fcmae_ft_in22k_in1k"
    )

    # `timm.create_model(name, pretrained=True, ...)` downloads and applies
    # the weights. The remaining kwargs are timm's regularisation options:
    #   num_classes=4    -> replace the head with a 4-class Linear
    #   drop_path_rate   -> stochastic depth: prob of randomly dropping
    #                       a whole residual branch in training. 0.3 is
    #                       a standard value for an 89M-parameter model on
    #                       ~8k training images.
    #   drop_rate        -> head dropout (before the final Linear).
    return timm.create_model(
        name,
        pretrained=True,
        num_classes=num_classes,
        drop_path_rate=0.3,
        drop_rate=0.3,
    )
```

4. Two-stage transfer learning

I train each backbone in two stages:

1. **Stage 1 (5 epochs)** — freeze the backbone, train only the new classifier head with Adam at LR 1e-3. This lets the head reach a reasonable starting point without disturbing the ImageNet features.
2. **Stage 2 (60 epochs)** — unfreeze the last stage of the backbone, switch to AdamW with cosine learning-rate schedule (5-epoch warmup, peak 1e-5, anneal to 1e-7). Class-weighted cross-entropy compensates for the imbalance (Stone is roughly 3.7× rarer than Normal).

I save the checkpoint with the best validation macro-F1, not the final epoch.

In [ ]:
```
# Section 4 — skeleton of my two-stage training loop.
# Full code lives in deep_learning/train.py — this is the demo cut.
```

```python
# `Adam` is the original adaptive-moment optimizer. We use it for stage 1
# (head only) — Adam works fine when no weight decay is needed.
# `AdamW` is Adam with decoupled weight decay (Loshchilov+Hutter 2019),
# which is the correct way to apply L2-like regularisation in adaptive
# optimizers. We use AdamW for stage 2.
from torch.optim import Adam, AdamW

# Learning-rate schedulers.
# `CosineAnnealingLR(optimizer, T_max, eta_min)` decays LR from its initial
#   value down to `eta_min` along the first half of a cosine curve.
#   T_max is the number of epochs in the cosine phase.
# `LinearLR(optimizer, start_factor, end_factor, total_iters)` linearly
#   ramps LR from start_factor * base_lr up to end_factor * base_lr over
#   `total_iters` epochs. We use it for the 5-epoch warmup.
# `SequentialLR(optimizer, schedulers, milestones)` chains schedulers — it
#   applies the first one until the milestone epoch, then switches to the
#   next one. We chain warmup then cosine.
from torch.optim.lr_scheduler import (
    CosineAnnealingLR,
    LinearLR,
    SequentialLR,
)


# ── Stage 1 ──────────────────────────────────────────────────────
# Freeze every backbone parameter so only the new head trains.
# `freeze_backbone(model)` is my helper (deep_learning/model.py) that sets
# `requires_grad = False` on backbone layers, `requires_grad = True`
# on the head. Frozen tensors have no gradient computation and contribute
# zero memory to autograd.
freeze_backbone(model)

# Build the optimizer only over the parameters that still require gradient.
# `model.parameters()` yields ALL parameters (frozen and trainable).
# The list comprehension `[p for p in ... if p.requires_grad]` filters to
# the head parameters only — Adam shouldn't update frozen weights.
# `lr=1e-3` is the head-warmup learning rate (high because we want the
# fresh head to reach a sensible starting point quickly).
optimizer = Adam(
    [p for p in model.parameters() if p.requires_grad],
    lr=1e-3,
)

# Train for 5 epochs. `range(5)` produces 0, 1, 2, 3, 4.
for epoch in range(5):
    # `train_one_epoch(...)` is my helper that runs one full pass over the
    # training DataLoader: forward, loss, backward, optimizer.step().
    train_one_epoch(model, train_loader, optimizer, loss_fn)

    # `evaluate_macro_f1(...)` is my helper that runs the model on the
    # validation DataLoader and returns sklearn macro-F1 over predictions.
    val_f1 = evaluate_macro_f1(model, val_loader)

    # Save the checkpoint only if it improved the validation metric.
    # `torch.save(state_dict, path)` serialises the model weights (not the
```

```
    # full nn.Module) to a .pt file. state_dict = dict of tensors.
    if val_f1 > best_val_f1:
        torch.save(model.state_dict(), ckpt_path)


# ── Stage 2 ──────────────────────────────────────────────────────
# Unfreeze the last `n_blocks` backbone stages so they get fine-tuned.
# n_blocks=1 for ConvNeXt V2 (bigger stages), n_blocks=2 for
# EfficientNet-B0 (smaller stages). Earlier stages stay frozen because the
# low-level ImageNet features transfer well as-is.
unfreeze_last_blocks(model, n_blocks=1)

# Switch to AdamW with stronger weight decay.
# `weight_decay=5e-2` is heavy regularisation for ConvNeXt V2's 89M params.
# For EfficientNet-B0 (5M params) I use 1e-4 — smaller models need less
# decoupled decay to avoid under-fitting.
# `lr=1e-5` is the peak LR after warmup; chosen 100x smaller than stage 1
# because we're now updating pretrained weights with small nudges.
optimizer = AdamW(
    [p for p in model.parameters() if p.requires_grad],
    lr=1e-5,
    weight_decay=5e-2,
)

# Linear warmup for 5 epochs: LR ramps from 1% of peak (0.01 * 1e-5 = 1e-7)
# up to 100% of peak (1e-5). Without warmup, the first big gradient step on
# the still-fragile head can destabilise the pretrained backbone.
warmup = LinearLR(
    optimizer,
    start_factor=0.01,
    end_factor=1.0,
    total_iters=5,
)

# Cosine anneal over the remaining 55 epochs: LR drops smoothly from peak
# 1e-5 down to eta_min=1e-7. Cosine is empirically kinder to pretrained
# weights than step decay or constant LR.
cosine = CosineAnnealingLR(optimizer, T_max=55, eta_min=1e-7)

# `SequentialLR` runs `warmup` first, then switches to `cosine` at epoch 5
# (the milestone). The optimizer's LR is updated every time we call
# `scheduler.step()` at the end of an epoch.
scheduler = SequentialLR(optimizer, [warmup, cosine], milestones=[5])

# Train for 60 epochs total in stage 2 (5 warmup + 55 cosine).
for epoch in range(60):
    train_one_epoch(model, train_loader, optimizer, loss_fn)
    val_f1 = evaluate_macro_f1(model, val_loader)
    if val_f1 > best_val_f1:
        torch.save(model.state_dict(), ckpt_path)
    # Advance the LR schedule one epoch.
    scheduler.step()
```

5. Inference and test-time augmentation

At inference I do two things beyond a plain forward pass:

1. **Save full softmax probabilities** (not just argmax predictions), because keeping probabilities lets me do paired McNemar's tests, soft-vote ensembles, and ROC-AUC without retraining.
2. **Horizontal-flip TTA**: predict on the original image, predict on the flipped image, average the two softmax vectors before argmax. Free accuracy boost at the cost of 2× inference time.

I tested rotation TTA too — it actually hurt accuracy, so I stuck with hflip only.

In [ ]:
```python
# Section 5 — inference with horizontal-flip TTA.
# Key idea: average softmax distributions, not argmaxes. Argmax-avg
# throws away the model's confidence — softmax-averaging keeps it.

import torch
import numpy as np
from PIL import Image

# `@torch.no_grad()` is a decorator that disables PyTorch's autograd engine
# inside the function. Saves memory + time; no gradient tracking
# gradients during inference. Equivalent to wrapping the body in a
# `with torch.no_grad():` block.
@torch.no_grad()
def predict_with_tta(model, image_uint8):
    # `model.eval()` switches BatchNorm/Dropout into evaluation mode:
    #   - Dropout becomes a no-op (don't zero activations at inference).
    #   - BatchNorm uses running statistics instead of batch statistics.
    # ConvNeXt uses LayerNorm so the BN part doesn't matter here, but
    # calling .eval() is the safe default for any model.
    model.eval()

    # Build the two TTA views.
    # `Image.fromarray(image_uint8, mode='L')` wraps the np array into
    # a PIL Image so we can use PIL's .transpose() for flipping. mode='L'
    # tells PIL it's single-channel grayscale.
    pil = Image.fromarray(image_uint8, mode="L")

    # `.transpose(Image.FLIP_LEFT_RIGHT)` returns a new PIL Image that is
    # the horizontal mirror. PIL's transpose constant FLIP_LEFT_RIGHT does
    # exactly what its name suggests — flips along the vertical axis.
    flip = pil.transpose(Image.FLIP_LEFT_RIGHT)

    # Apply the standard val transform (resize + ImageNet-normalise +
    # 3-channel-replicate) to each view, then add a batch dimension with
    # `.unsqueeze(0)` so the model sees a (1, 3, H, W) tensor.
    # `.to(device)` ships the tensor to the GPU (cuda/mps) if available.
    x_orig = transform(np.asarray(pil)).unsqueeze(0).to(device)
    x_flip = transform(np.asarray(flip)).unsqueeze(0).to(device)

    # Forward pass on each view. `model(x)` returns raw logits of shape
    # (batch, num_classes) = (1, 4).
    # `torch.softmax(logits, dim=1)` converts logits to probabilities along
```

```
    # the class dimension (each row sums to 1).
    p_orig = torch.softmax(model(x_orig), dim=1)
    p_flip = torch.softmax(model(x_flip), dim=1)

    # Average the two probability vectors -- heart of soft-vote TTA.
    p_avg = (p_orig + p_flip) / 2

    # `.argmax(dim=1)` -> class index with the highest averaged prob.
    # `.item()` extracts a Python scalar from a 0-dim tensor.
    # `.cpu().numpy()` moves the tensor back to CPU and converts
    # it to a numpy array so the caller can pickle or use it w/ sklearn.
    return p_avg.argmax(dim=1).item(), p_avg.cpu().numpy()
```

6. Headline numbers (clean held-out test set, n = 1,888)

I report macro-F1 because the class distribution is imbalanced (Stone ~13 %, Normal ~41 %) — accuracy would over-weight the easy majority class.

In [ ]:

```
# Section 6 — load the four pre-computed result JSONs and display them.

# `json` is Python's built-in JSON parser; loads our .json files
# my predict scripts saved.
import json
# `Path` again for path joining.
from pathlib import Path
# `pd` for tabular display.
import pandas as pd

# `RESULTS` points at the Results/ directory inside the project root.
# (`ROOT` was set in Section 2.)
RESULTS = ROOT / "Results"

# `models` is a dict mapping display name -> relative path to the JSON file
# under Results/. Using a dict (vs two parallel lists) makes the loop below
# read more naturally.
models = {
    "EfficientNet-B0":
        "dl_run_full/dl_results.json",
    "EfficientNet-B0 + hflip TTA":
        "dl_run_full_tta_hflip/dl_results.json",
    "ConvNeXt V2 Base":
        "convnextv2_full_run/dl_results.json",
    "ConvNeXt V2 Base + hflip TTA":
        "convnextv2_full_run_tta_hflip/dl_results.json",
}

# `rows` is a list of dicts; each dict becomes one DataFrame row.
rows = []

# `.items()` iterates over (key, value) pairs of the dict.
for name, rel in models.items():
    # `(RESULTS / rel)` joins the result-dir Path with the relative path.
```

```
    # `.read_text()` returns the file contents as a single string.
    # `json.loads(s)` parses a JSON string into a Python dict.
    r = json.loads((RESULTS / rel).read_text())

    # Build one row of the table. `.append(dict)` adds to the list.
    # `round(x, 4)` truncates the float to 4 decimal places for display.
    rows.append({
        "Model": name,
        "Accuracy": round(r["accuracy"], 4),
        "Macro-F1": round(r["macro_f1"], 4),
        # `r["per_class"]["Stone"]["f1"]` is a nested-dict lookup into the
        # JSON's per-class breakdown for the Stone class's F1 score.
        "Stone F1": round(r["per_class"]["Stone"]["f1"], 4),
    })

# `pd.DataFrame(rows)` builds a DataFrame from a list of dicts.
# In a Jupyter cell, evaluating a DataFrame as the last expression
# automatically calls its HTML representation for nice rendering.
pd.DataFrame(rows)
```

ConvNeXt V2 + TTA is my best single deep-learning model at macro-F1 0.837. EffNet-B0 + TTA is 0.795. For comparison, Anthony's classical SVM reaches macro-F1 0.909 on the same test set — classical wins on this clean benchmark.

7. Per-class results — Stone is the universal weak class

Stone is the lowest-F1 class for every model I trained. The dominant error type is `Stone → Normal`, because small focal calcifications get diluted when a whole-slice CNN classifier pools over the entire image.

In [ ]:
```
# Section 7 — confusion matrix for my best DL model.

# `numpy` for converting JSON list-of-lists to a 2D array.
import numpy as np
# `matplotlib.pyplot` is the standard plotting API; aliased as `plt`.
import matplotlib.pyplot as plt

# Load the same result JSON used in Section 6 (just the ConvNeXt+TTA one).
r = json.loads(
    (RESULTS / "convnextv2_full_run_tta_hflip"
     / "dl_results.json").read_text()
)
# `r["confusion_matrix"]` is a 4x4 list-of-lists; we convert to numpy so
# `.max()`, indexing, and matplotlib's `.imshow()` work cleanly.
cm = np.array(r["confusion_matrix"])

# Class labels in the canonical project order.
classes = r["classes"]

# `plt.subplots(figsize=(W,H))` creates one figure and one axis. size is
# in inches (5 wide x 4 tall).
```

```
fig, ax = plt.subplots(figsize=(5, 4))

# `ax.imshow(cm, cmap="Blues")` draws the confusion matrix as a heatmap.
# `cmap="Blues"` maps low values to white, high values to dark blue.
# Returns the AxesImage handle (`im`) for an optional colorbar later.
im = ax.imshow(cm, cmap="Blues")

# Replace the integer tick labels with the class names.
# `rotation=35` rotates by 35 degrees; `ha="right"` aligns the label so the
# right end sits at the tick (avoids overlap).
ax.set_xticks(range(4), classes, rotation=35, ha="right")
ax.set_yticks(range(4), classes)
ax.set_xlabel("Predicted")
ax.set_ylabel("True")
ax.set_title("ConvNeXt V2 + TTA — clean test")

# Annotate each cell of the matrix with its count.
# Outer loop over rows (true class), inner loop over columns (predicted).
for i in range(4):
    for j in range(4):
        # Choose readable text colour: white on dark cells, black on light.
        # `cm.max() * 0.55` is the threshold; >55% of max gets the
        # matrix maximum gets white text (so it's visible on dark blue).
        color = "white" if cm[i, j] > cm.max() * 0.55 else "black"

        # `ax.text(x, y, s, ...)` places text. Coords are data units
        # (column index, row index). `ha`/`va` are horizontal/vertical
        # alignment so the text is centered in the cell.
        ax.text(j, i, str(cm[i, j]), ha="center", va="center", color=color)

# Auto-adjust subplot params to prevent label clipping.
plt.tight_layout()
# Render the figure inline in the notebook.
plt.show()
```

8. The leakage diagnostic — my headline finding

While exploring the dataset I noticed our DL models were hitting accuracies that looked too good (EfficientNet-B0 at 0.94, ConvNeXt V2 at 1.000). I wrote a small script that MD5-hashes every standardised pixel array and compares the train, val, and test sets for collisions.

I found **5.1 % of test images were bit-for-bit identical to training images** — 96 out of 1,867 hash collisions. The dataset author independently confirmed the duplicates two days later and released a deduplicated version, which is the dataset I use everywhere else in this notebook.

The leakage gap is the most important number in the project:

- **EfficientNet-B0**: dropped 17.3 percentage points (0.940 → 0.768)
- **ConvNeXt V2 Base**: dropped 21.4 percentage points (1.000 → 0.822)

- **Classical SVM**: dropped only 8.4 percentage points (0.993 → 0.909)

Deep networks were memorising the duplicated images; classical handcrafted features were not. That's the strongest evidence I have that prior reported 99 %+ accuracies on this benchmark are inflated.

In [ ]:
```
# Section 8 — the leakage detection function. This is the heart of my
# headline finding.

# `hashlib` is Python's built-in crypto-hash library; we use MD5
# because it's fast and has near-zero collision prob for distinct
# (~10^-38), and gives a 32-character hex string per image — perfect for
# set-intersection comparisons.
import hashlib
# `Counter` from collections counts repeated occurrences of items in a
# sequence; used in the actual script to find within-split dups.
from collections import Counter

def md5_image(path):
    # Critical design choice: I hash the STANDARDISED pixel buffer, not the
    # raw JPEG bytes. Two JPEGs with different compression settings
    # but identical pixel content still collide. Two identical files with
    # different EXIF metadata also collide. We're checking *image content
    # equality after our preprocessing*, which is what actually matters for
    # train/test contamination.
    img = load_image(path)   # the shared loader from Section 1

    # `img.tobytes()` returns the raw bytes of the underlying numpy buffer.
    # For a (256, 256) uint8 array this is exactly 65,536 bytes.
    # `hashlib.md5(b)` returns a hash object; `.hexdigest()` returns its
    # 32-character lowercase hex representation, e.g. "0c4a9b3e..." .
    return hashlib.md5(img.tobytes()).hexdigest()

# In the real diagnostic I ran this in parallel over ~12k images via
# joblib.Parallel, then intersected the per-split hash sets to count
# collisions. Conceptually:
#
#     train_hashes = {md5_image(p) for p in train_paths}  # set comp.
#     test_hashes  = {md5_image(p) for p in test_paths}
#     duplicates   = train_hashes & test_hashes        # set intersection
#                                                       # `&` on sets =
#                                                       # common elements
#
# `{ ... for ... in ... }` is a set comprehension. Sets give O(1) average
# membership and intersection.
#
# Result on the original Islam et al. 2022 release:
#   96 / 1,867 test images had a bit-identical training-set sibling = 5.1%
#
# That 5.1% number is what I report as my headline finding.
```

9. Summary of what I built

In bullet form (this is roughly the closing slide of my demo):

- **Shared layer** I co-designed with Anthony: `load_image()`, `load_split()`, `bootstrap_ci()`, paired McNemar's test.
- **My deep-learning code**: EfficientNet-B0 and ConvNeXt V2 builders, two-stage transfer learning with cosine LR + warmup, class-weighted CE, inference with hflip TTA, multi-seed ensembling, Grad-CAM interpretation.
- **My most novel finding**: the MD5 leakage diagnostic — discovered 5.1 % bit-identical train↔test duplication that previously-published 99 %+ accuracies were silently relying on.
- **Honest conclusion**: on the cleaned benchmark, Anthony's classical SVM beats both of my deep-learning models by 7 to 14 percentage points. Deep networks aren't free-lunch superior here; they're more leakage-sensitive and have lower sample efficiency.

The full technical detail and every artefact lives in `James_Wu_Deep_Learning_Technical_Contribution.ipynb` and the `Planning/` vault. This summary notebook is just the demo-friendly cut.